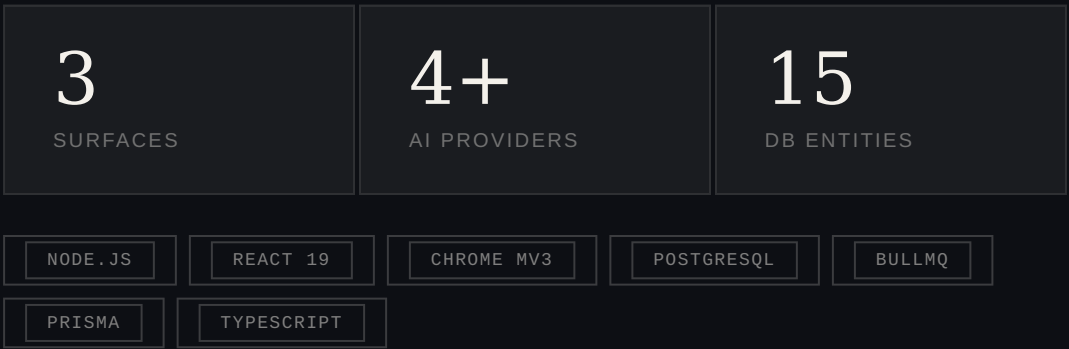


IntentFlow

A full-stack AI SEO Intelligence Platform that captures invisible user behavior from AI chat providers and materializes it into structured, actionable campaign data.



Bridging the *visibility gap* in AI-driven search

Traditional SEO tools monitor Google searches. But as users migrate to conversational AI — ChatGPT, Claude, Perplexity, Grok — those queries and the sites AI models cite are entirely invisible to standard analytics.

IntentFlow bridges this gap by intercepting live AI response streams, parsing them into hierarchical campaign trees, and enriching them with third-party SEO data from SEMrush and Ahrefs — all in real time.

The result: every AI conversation a user has becomes a structured, deduplicated, version-controlled SEO intelligence record that operators can explore, compare over time, and act on.

The Core Problem

Millions of searches now happen inside AI chatbots. These queries — and the websites AI recommends — generate zero signal in Google Analytics, SEMrush, or any traditional tool.

- Traditional tools see nothing
- Intent stays invisible
- SEO opportunity is missed

IntentFlow makes this invisible layer visible — and actionable.

Three independently deployable *surfaces*

IntentFlow is a monorepo with three surfaces communicating over HTTPS via REST APIs and JWT authentication, backed by a shared PostgreSQL + Redis data layer.

SURFACE 01 · CHROME
MV3

Extension

Injects MAIN-world content scripts into AI provider pages to intercept SSE/streaming responses in real time. Side-panel UI for live visualization.

Chrome MV3

CRXJS

React

Vite

SURFACE 02 · NODE.JS

Backend API

Ingests raw events, materializes campaign trees using deterministic SHA-1 hashing, orchestrates BullMQ enrichment jobs, enforces RBAC.

Express.js

Prisma

BullMQ

JWT

Zod

SURFACE 03 · REACT
19

Web Dashboard

Campaign graph exploration (XYFlow), analytics (Recharts), admin controls, onboarding flow, and lead intelligence scoring views.

React 19

Vite

XYFlow

Recharts

Tailwind

LAYER	TECHNOLOGY
Backend Runtime	Node.js 20+ · TypeScript
Web Framework	Express.js 4
Database	PostgreSQL via Prisma ORM
Queue / Jobs	BullMQ on Redis
Authentication	JWT access tokens + DB-backed refresh tokens
Frontend	React 19, Vite, Tailwind CSS, Shadcn UI / Radix UI
Visualization	XYFlow (campaign trees) · Recharts (analytics)
Extension Build	Chrome MV3 · CRXJS plugin · Vite
Validation	Zod (DTO schema validation)
External APIs	OpenAI GPT · SEMrush · Ahrefs

Engineering decisions that *scale*

3.1

Layered Architecture (N-Tier)

Strict 4-tier separation: Routes to Controller to Service to Repository. Swapping PostgreSQL for another database only touches the Repository layer.

```
Routes → Controller → Service →
Repository
Each layer only speaks to the one
below it
```

3.2

Multi-Tenancy

Every database record carries a `tenant_id`. Row-level isolation is enforced in the Repository layer — no tenant can ever access another's data, even on the same server.

```
WHERE tenant_id = ? on every
query
Enforced in Repository, not
business logic
```

3.3

Event-Driven / Queue-Based

Heavy enrichment tasks (SEMrush, Ahrefs, NLP) are pushed to BullMQ queues, never blocking the HTTP response. Producer-Consumer pattern throughout.

```
ingestTurn() → push job → return
201
Worker processes async in
background
```

3.4

Idempotency

`ingestTurn()` checks `request_id` + `turn_exchange_id` before writing. Duplicate events from network retries return the existing tree instead of creating duplicates.

```
if (exists) return existing;
else create and return new;
```

3.5

Deterministic Hashing

Internal IDs use SHA-1 hashes of content rather than random UUIDs. The same prompt from the same provider always produces the same hash — enabling reliable deduplication.

```
SHA1(provider + turn_id + query +
site)
→ always same result_ref for same
content
```

3.6

Caching + RBAC

SEMrush/Ahrefs results cached in `SemrushSnapshot` with 24-hour TTL. Dual-role RBAC (App Role: admin/user · Tenant Role: owner/member) enforced via JWT middleware on every route.

```
authMiddleware →
requireRole('admin')
→ requireAccountRole('owner')
```

Full OOP in *TypeScript*

IntentFlow uses TypeScript's complete OOP feature set — not just as a style choice, but because each pattern directly solves a specific architectural problem.

ABSTRACTION

Abstract Base Classes

Three abstract classes — `BaseController`, `BaseService`, `BaseRepository` — define structural contracts for the entire backend. All modules extend these.

```
class CampaignService
  extends BaseService
```

ENCAPSULATION

Private Methods

`CampaignService` hides `resolveVersion()`, `toVersionSummary()`, `toChatThreadSummary()` as private methods. Public API stays simple.

```
private resolveVersion()
private listExecutedPromptsForVersion(versionId)
```

INHERITANCE

Class Hierarchy

Concrete classes extend abstract bases, inheriting shared utility methods. Adding a utility to `BaseService` automatically benefits all services.

```
CampaignController
  extends BaseController
AuthRepository
  extends BaseRepository
```

POLYMORPHISM

Generics + Runtime

`ApiResponse.success` returns different shapes per type `T`. Stream interception uses runtime polymorphism — same interface, four different provider implementations.

```
ApiResponse.success(Campaign)(data)
ApiResponse.success(User[])(data)
```

SINGLETON

Single Instances

Services, controllers, and repositories are instantiated once at module load. Prevents redundant DB connections and ensures consistent state.

```
export const campaignService =
  new CampaignService(campaignRepository)
```

DI + FACTORY

Injection and Factory

`CampaignService` receives its repository via constructor injection — enabling unit testing with mock repos. `ApiResponse` acts as a factory for all response envelopes.

```
constructor(private repo: CampaignRepository) {
  ApiResponse.success(data, 'Created')
}
```

STRATEGY PATTERN

Per-Provider Stream Interception

Each AI provider (ChatGPT, Claude, Perplexity, Grok) has its own content script implementing the same event-posting contract but using a completely different interception strategy — SSE parsing, HTTP chunked response parsing, DOM event listening, etc. The background service worker acts as the context that delegates to whichever strategy is active.

```
chatgptStreamContent.ts · claudestreamContent.ts · perplexityStreamContent.ts ·
grokStreamContent.ts — all emit AI_SEO_* typed events via the same contract
```

15 entities around a *self-referencing tree*

PostgreSQL managed via Prisma ORM. The central data model is a PromptNode adjacency list — a single table representing an unbounded prompt to subquery to site hierarchy.

User Platform user — email, password hash, app role, lead score data.	Tenant Organisation/account boundary. All data is scoped to a tenant.	TenantMember Join table: User and Tenant with role (owner / member).
Campaign An SEO campaign linked to a Tenant and optionally a Domain.	CampaignVersion Point-in-time snapshot of a campaign. Can be archived and refired.	CaptureSession One AI chat conversation linked to a CampaignVersion.
CaptureTurn One prompt-response exchange within a session. Stores raw_event_json.	PromptNode (star) Self-referencing tree node. Types: prompt, subquery, site, generated.	SemrushSnapshot Cached keyword and traffic data from SEMrush / Ahrefs. TTL: 24h.
LeadSignal Buying-intent signal extracted from a capture turn.	Domain A tracked website domain with scrape status and context JSON.	GenerationRun Tracks an AI suggestion generation job (queued/processing/done).

The Campaign Tree — Adjacency List

```
// PromptNode — self-referencing, parent_id points to another PromptNode
• prompt "best SEO tools 2026" parent_id = null (root)
  └─ subquery "top seo software" parent_id = prompt.id
    └─ site ahrefs.com parent_id = subquery.id
    └─ site semrush.com parent_id = subquery.id
    └─ generated "compare seo pricing" source = 'suggestion'
// All IDs are deterministic SHA-1 hashes — not random UUIDs
```

Database Enums

NodeType prompt · subquery · site · generated	AiChatProvider chatgpt · claude · gemini · perplexity · grok · unknown
CampaignVersionStatus draft · active · archived	AppRole admin · user
RunStatus queued · processing · completed · failed	CaptureStatus listening · intercepting · analyzing · generating · complete · failed

06 · DATA FLOW

Extension to Backend to *Dashboard*

The full end-to-end pipeline from a user typing a prompt in ChatGPT to a fully enriched campaign tree visible in the web dashboard.

1

User opens ChatGPT / Claude / Perplexity

The extension's MAIN-world script is injected by Chrome via manifest.json — bypassing Content Security Policy. It patches the browser's native fetch/XHR.

```
chatgptStreamMain.ts · world: MAIN
```

2

Live stream is intercepted and parsed

The MAIN-world script intercepts SSE/streaming responses. Extracts: the user's prompt, any internal search queries the AI ran, and websites cited in the response.

```
streamMain.ts → window.postMessage → streamContent.ts
```

3

Data relayed to background service worker

The isolated-world content script relays parsed events to the background worker via chrome.runtime ports. Background batches events and calls the backend API.

```
chrome.runtime.connect() → background.ts
```

4

POST /campaigns/:id/ingest-turn

Structured JSON payload sent to backend with JWT auth. Zod validates the DTO. Controller delegates to CampaignService.ingestTurn().

```
authMiddleware → CampaignController → CampaignService
```

5

Campaign tree materialized in PostgreSQL

Service normalizes data, resolves the active CampaignVersion, computes deterministic SHA-1 hashes, and upserts PromptNode records via the Repository.

```
CampaignService → CampaignRepository → Prisma → PostgreSQL
```

6

Enrichment jobs queued asynchronously

SEMrush and Ahrefs jobs are pushed to BullMQ — the HTTP response returns immediately. Workers process in parallel, store SemrushSnapshot records with 24-hour TTL.

```
BullMQ → semrush.worker.ts → SemrushClient → API
```

7

Lead intelligence extracted (fire-and-forget)

LeadIntelligenceService.extractSignalsFromTurn() runs asynchronously — scoring buying intent, product interest, and behavioral signals from the raw prompt text.

```
LeadIntelligenceService → LeadSignal table
```

8

Dashboard renders the live campaign tree

Extension side-panel and web dashboard fetch the enriched tree via GET /campaign/:id/active-tree. XYFlow renders the interactive graph; Recharts shows analytics.

```
React Dashboard → XYFlow graph · Recharts analytics
```

Built by *six*



Sibtain Ahmed Qureshi

PROJECT LEAD AND ARCHITECT

- Designed monorepo structure and surface interaction model
- Built the deterministic materialization engine (SHA-1 tree mapping)
- Architected the Prisma schema and PostgreSQL migration strategy



Abhishek Verma

STRATEGY AND SYSTEMS DESIGN

- Conceptualized lead intelligence framework and intent scoring models
- Established Agent Context governance for AI-assisted development
- Designed multi-tenant RBAC security architecture



Amogha Raj Sandur

DEVOPS AND QUALITY ASSURANCE

- Managed CI/CD alignment and task-to-milestone mapping
- Led monorepo refactoring for modularity and low technical debt
- Architected handoff documentation and progress tracking standards



Mohammed Yaseen

FULL-STACK ENGINEERING

- Built Mock Server and testing ecosystem for provider-free local dev
- Led cross-surface sync between extension side-panel and web dashboard
- Implemented onboarding and analytics flows

What comes *next*

Server-Driven Replay and Automation

Move campaign refiring from the extension to a fully server-side model — enabling scheduled background re-runs and monitoring without requiring an active browser session.

NEAR-TERM

Advanced Lead Scoring and CRM Integration

ML models refined on historical interaction data. Automated export pipelines to Salesforce and HubSpot via custom webhooks.

MID-TERM

Enterprise Observability and Scaling

OpenTelemetry integration for end-to-end event tracing. Scaling the Redis job queue to handle thousands of concurrent enterprise ingestion streams.

MID-TERM

Pilot Readiness and Commercial Launch

Multi-tenant billing infrastructure, SOC2 compliance readiness, and high-availability deployment hardening for public pilot launch.

LONG-TERM